

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

MANEJO DE BLOQUEOS Y ACCESO CONCURRENTES

Cuando varias personas intentan modificar la misma información en una base de datos al mismo tiempo, pueden ocurrir problemas de concurrencia. Para evitar errores y mantener la integridad de los datos, los sistemas de gestión de bases de datos (SGBD) utilizan bloqueos (locks).

¿QUÉ ES UN BLOQUEO?

Un bloqueo es una restricción temporal que evita que dos transacciones accedan al mismo dato al mismo tiempo, por ejemplo, es lo que sucede en un documento compartido en Google Docs, donde si una persona está editando, los demás solo pueden ver los cambios, pero no modificarlos, cuando el usuario termina de editar y guarda, el documento vuelve a estar disponible, estas acciones son similares a lo que hace un SGBD con los bloqueos.

CASOS TÍPICOS QUE PUEDEN SUCEDER SIN LOS BLOQUEOS CORRECTOS

Lecturas sucias (Dirty Reads): Ocurre cuando una transacción lee datos que otra transacción aún no ha confirmado (puede ser que luego se deshagan con ROLLBACK).

Lecturas no repetibles (Non-Repeatable Reads): Una consulta obtiene diferentes resultados en dos ejecuciones dentro de la misma transacción porque otra transacción modificó los datos entre ambas lecturas.

Fantasmas (Phantom Reads): Ocurre cuando una transacción consulta un conjunto de datos varias veces y, entre una lectura y otra, otra transacción inserta o elimina registros, cambiando el resultado.

TIPOS DE BLOQUEOS EN SQL

Los SGBD usan diferentes tipos de bloqueos para controlar el acceso concurrente, estos son el **bloqueo exclusivo** y el **bloqueo compartido**:

Bloqueo Exclusivo (Exclusive Lock - X Lock) que se usa cuando una transacción **modifica un dato** y mientras está activo, **nadie más** puede leer ni modificar ese dato.

Bloqueo Compartido (Shared Lock - S Lock), que se usa cuando una transacción **solo lee** un dato y varias transacciones pueden leer al mismo tiempo, pero **ninguna puede modificar** hasta que se liberen los bloqueos.

Por ejemplo, **un usuario** quiere **actualizar** el saldo de una cuenta, y **otro usuario** intenta **leer** el saldo **al mismo tiempo**, sin bloqueos, podría ocurrir una lectura inconsistente, para evitarlo, aplicamos un **bloqueo exclusivo** antes de actualizar, utilizando el comando **FOR UPDATE** que nos permite bloquear la fila para que nadie más la modifique mientras la transacción está en curso y si otro usuario intenta leer la misma cuenta, deberá esperar a que se complete la transacción. ¿Lo llevamos a la práctica?

```
START TRANSACTION;  
// Bloquear la fila de la cuenta 1 para evitar accesos simultáneos //  
SELECT saldo FROM Cuentas WHERE id_cuenta = 1 FOR UPDATE;  
// Modificar el saldo //  
UPDATE Cuentas SET saldo = saldo - 100 WHERE id_cuenta = 1;  
COMMIT;
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

¡MUY IMPORTANTE!

¿Qué no debemos olvidar? Los bloqueos evitan inconsistencias cuando varias transacciones acceden a los mismos datos.

¿Algo más? Si, existen diferentes tipos de bloqueos según si se va a leer o modificar la información, por eso resulta extremadamente importante configurar correctamente los bloqueos para evitar problemas como bloqueos prolongados o bloqueos muertos (deadlocks).

EJERCITAMOS CON BLOQUEOS EN SQL

Vamos a experimentar con el uso de bloqueos en SQL para evitar problemas de concurrencia cuando varias transacciones acceden a los mismos datos en una reserva de entradas para un recital, donde queremos asegurarnos de que dos clientes no compren la última entrada disponible al mismo tiempo y para solucionar un posible problema usaremos los bloqueos en SQL que nos permitirán controlar el acceso concurrente a la base de datos.

CREAMOS LA BASE DE DATOS Y LE CARGAMOS ALGUNOS DATOS

En este caso, hay **solo una entrada disponible** para el concierto.

```
CREATE DATABASE Ticketec;
```

```
USE Ticketec;
```

```
CREATE TABLE Entradas (
```

```
    id_entrada INT PRIMARY KEY AUTO_INCREMENT,
```

```
    evento VARCHAR(100),
```

```
    cantidad_disponible INT
```

```
);
```

```
INSERT INTO Entradas (evento, cantidad_disponible) VALUES
```

```
('Concierto Rock', 1);
```

AHORA VAMOS A SIMULAR UNA COMPRA SIN BLOQUEO

Cuando dos usuarios intentan comprar la entrada al mismo tiempo, para ello vas a tener que realizar la consulta en dos sesiones diferentes (dos clientes):

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock';
```

Si ambos usuarios ven que hay una entrada disponible y la compran al mismo tiempo, la base de datos podría permitir vender más de una entrada, generando un error.

¿CÓMO LO SOLUCIONAMOS?

Aplicando un bloqueo para evitar la sobreventa, para esto debemos bloquear la fila antes de modificarla.

Prestamos mucha atención porque aquí vamos a ejecutar código en sesiones diferentes, pero primeramente ¿Cómo usamos sesiones diferentes en SQL? A continuación lo vemos.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****ACCESO CONCURRENTE, INICIO DE SESIONES DIFERENCIADAS EN SQL**

Para simular el acceso concurrente a la base de datos, necesitamos **dos sesiones separadas** de SQL, lo cual podemos hacerlo de varias maneras, usando MySQL en un servidor local o en un servidor remoto, MySQL Workbench, phpMyAdmin, PostgreSQL o SQL Server.

USANDO MYSQL EN UN SERVIDOR LOCAL O REMOTO

Primero abrimos **dos terminales** (cmd, bash o PowerShell) y nos conectamos a MySQL en ambas, también podemos abrir **dos ventanas del cliente MySQL Workbench o phpMyAdmin y abrir dos pestañas o conexiones** al mismo servidor.

Con PostgreSQL o SQL Server, podemos usar dos instancias separadas de psql o SSMS (SQL Server Management Studio).

PRACTIQUEMOS

Primeramente crearemos una tabla a la que le vamos a insertar datos iniciales

ATENCIÓN: En **ambas sesiones**, ejecuta la creación de la tabla y los datos iniciales:

```
CREATE TABLE Entradas (  
    id_entrada INT PRIMARY KEY AUTO_INCREMENT,  
    evento VARCHAR(100),  
    cantidad_disponible INT  
);  
  
INSERT INTO Entradas (evento, cantidad_disponible) VALUES  
( 'Concierto Rock', 1);
```

EJECUTAR UNA COMPRA SIN BLOQUEOS (PARA VER EL PROBLEMA)

En la primera sesión (Cliente 1), ejecutar:

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock';
```

En la segunda sesión (Cliente 2), ejecutar lo mismo:

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock';
```

RESULTADO: Ambos verán que hay 1 entrada disponible.

Ahora, ambos intentan comprar la entrada al mismo tiempo:

En Cliente 1:

```
UPDATE Entradas SET cantidad_disponible = cantidad_disponible - 1 WHERE evento = 'Concierto Rock';
```

En Cliente 2:

```
UPDATE Entradas SET cantidad_disponible = cantidad_disponible - 1 WHERE evento = 'Concierto Rock';
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

¿QUE PASA? TENEMOS UN PROBLEMA

Ambos compradores piensan que hay una entrada disponible y la compran al mismo tiempo, lo que podría llevar a un saldo negativo (cantidad_disponible = -1).

EJECUTAR LA COMPRA CON BLOQUEOS (SOLUCIÓN CORRECTA)

Ahora aplicamos bloqueos para evitar la sobreventa.

En Cliente 1, ejecutar la compra con un bloqueo:

```
START TRANSACTION;
```

```
// Bloquear la fila de la entrada para evitar acceso simultáneo //
```

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock' FOR UPDATE;
```

```
// Verificar disponibilidad y actualizar //
```

```
UPDATE Entradas SET cantidad_disponible = cantidad_disponible - 1
```

```
WHERE evento = 'Concierto Rock' AND cantidad_disponible > 0;
```

```
// Confirmar la compra //
```

```
COMMIT;
```

¿QUE SUCEDIÓ?

El comando FOR UPDATE bloquea la fila, evitando que otro usuario la modifique hasta que termine la transacción y luego el comando COMMIT guarda los cambios.

CONTINUEMOS PROBANDO LA CONCURRENCIA

En Cliente 2, intentar ejecutar la misma compra:

```
START TRANSACTION;
```

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock' FOR UPDATE;
```

```
UPDATE Entradas SET cantidad_disponible = cantidad_disponible - 1
```

```
WHERE evento = 'Concierto Rock' AND cantidad_disponible > 0;
```

```
COMMIT;
```

¿QUE SUCEDIÓ?

El **Cliente 2** tendrá que **esperar a que Cliente 1 termine la transacción** antes de poder continuar y como **Cliente 1** ya compró la única entrada, la consulta de Cliente 2 **fallará** (porque cantidad_disponible = 0).

ACTIVIDAD DE CIERRE INDIVIDUAL DE LA PRACTICA INICIAL

Luego de realizar esta actividad concurrente ¿Qué pasó cuando intentamos comprar sin bloqueos? ¿Cómo el bloqueo evitó que se vendieran más entradas de las disponibles? ¿En qué otras situaciones es importante usar bloqueos en bases de datos?

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****PRACTICA CONCRETA DE APLICACIÓN**

Ahora que ya sabemos crear y manejar sesiones simultaneas, volvemos al planteo inicial que veíamos antes, cerrada la ejercitación anterior, abrimos dos sesiones nuevas y:

En la **primera sesión (Cliente 1)** utilizaremos el comando **FOR UPDATE** que bloqueará la fila, impidiendo que otro usuario la modifique hasta que se complete la transacción y la condición `cantidad_disponible > 0` evita que la cantidad se vuelva negativa.

```
START TRANSACTION;
```

```
// Bloquear la fila para evitar que otro cliente la modifique //
```

```
SELECT cantidad_disponible FROM Entradas WHERE evento = 'Concierto Rock' FOR UPDATE;
```

```
// Verificar disponibilidad //
```

```
UPDATE Entradas SET cantidad_disponible = cantidad_disponible - 1
```

```
WHERE evento = 'Concierto Rock' AND cantidad_disponible > 0;
```

```
// Confirmar la compra //
```

```
COMMIT;
```

AHORA PROBEMOS LO MISMO EN LA SEGUNDA SESION

Replica el código inicial y luego presta atención ¿NOTASTE QUE SUCEDIÓ? Si intentás ejecutar lo mismo en la **segunda sesión (Cliente 2)**, te vas a dar cuenta que debe esperar a que la primera sesión termine.

ACTIVIDAD DE CIERRE INDIVIDUAL

Finalmente, luego de toda la práctica ¿Qué sucedió cuando intentaste ejecutar la compra en dos sesiones a la vez sin bloqueo? ¿Cómo evitó el bloqueo que se vendieran más entradas de las disponibles? ¿En qué otros escenarios crees que los bloqueos son importantes?